
Android and Mobile Auditing Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *November 28th, 2019*

Introduction

1. Android is a Linux kernel with a userspace and a vendor supply chain bolted on top. Everything from the Linux kernel module still applies; what changes is the shape of the attack surface, who wrote it, and which policy layer sits between an attacker and the kernel.
2. The module is organized as two question sets. Part I is the Linux knowledge the rest of the course assumes but never states: how the kernel exposes process state, how mappings are created, how one process manipulates another, and how the mandatory access control layer is configured. Part II is Android-specific: how a phone is rooted and debugged, what an attacker with (and without) adb can reach, and how the crash-reporting pipeline works.
3. Read this module after the Linux kernel auditing module. The kernel module covers the generic driver surface; this module covers the vendor-written extensions of that surface and the practical mechanics of getting a shell on a phone in the first place.

Part I: Linux-Specific Knowledge for VR

1. **What does the `/proc/self/maps` file tell you and why is it important? Also, how is it implemented in the kernel?**
 - (a) It can be opened and read as a regular file, and it returns a table containing the virtual memory ranges (VMAs) mapped for the current process: the exploit developer's map of protections and layout (where libc is, where the stack and heap are, what is executable). Each line is

Listing 1: One line of the process memory map, as returned by the file

```
1 7f2c3a400000-7f2c3a5b1000 r-xp 00000000 08:01 1234 /system/lib64/libc
```

that is: start-end, permission bits (r, w, x plus p for private or s for shared), the offset into the file the mapping begins at, device and inode, and the pathname.

- (b) Importance: one leaked pointer inside a mapping plus the range itself gives the address of every other symbol in that mapping. The file also reveals the ASLR slide (the load address of the executable versus its link-time address), which regions are writable, which are executable, and whether any single region is both (RWX). That is the raw material of every address-dependent exploit primitive.

- (c) Pathnames worth recognizing: [stack] (the main thread stack, grown down), [heap] (the brk-managed region), [vdso] / [vvar] (kernel-supplied pages mapped into every process), [vsyscall] (the fixed-address x86_64 compatibility page), [uprobes] (instrumentation), anonymous kernel files such as anon_inode:binder or anon_inode:ashmem, and any path ending in (deleted) — a file unlinked while still mapped, which is a common way for a dropped payload or a replaced library to hide.
- (d) It is implemented as a sequence (seq_file) file type. The REG("maps", S_IRUGO, proc_pid_maps) registration lives in fs/proc/base.c of the Linux source tree, and the operations it names (maps_open, show_map_vma, m_start/m_next/m_stop) are implemented in fs/proc/task_mmu.c. Reads walk the VMA list hanging off the target's struct mm_struct (mm->mmap) while holding mmap_lock.
- (e) Reading another process's /proc/<pid>/maps requires PTRACE_MODE_READ, enforced by ptrace_may_access(). That is the same permission check that gates /proc/<pid>/mem and process_vm_readv, so an Android app cannot read another app's layout even though the DAC mode bits are world-readable.
- (f) Related interfaces in the same family: /proc/self/smaps (per-mapping resident, dirty and swap accounting, plus the kernel's page size), /proc/self/smaps_rollup (the aggregate), /proc/self/map_files (symlinks from each range to its backing file) and /proc/self/numa_maps. When maps is insufficient, smaps shows whether a page is shared, which decides whether a heap groom can touch it.

2. What are the uses of mmap? What are some bugs regularly found in mmap?

- (a) Anonymous allocations: mmap(NULL, len, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS) is the general-purpose allocator for large blocks and for thread stacks; glibc itself uses it for large malloc requests. MAP_FIXED turns it into a deterministic-layout primitive: the caller names the exact address, which is how an exploit places a fake structure or a ROP stack where it needs it.
- (b) File-backed mappings: a file is mapped without syncing all of its contents into physical memory at once; pages are faulted in on demand and shared through the page cache. Dynamic-linker loading of every PT_LOAD segment is itself a sequence of mmap calls, which is why a leak of one libc address yields the whole library.
- (c) Device memory exported to userspace: drivers (notably GPUs) use their file_operations.mmap handler to export or import virtual pages from the device. This is the direction that matters for exploitation: kernel memory is being mapped into an attacker's address space, so the bounds checks on vma->vm_pgoff and the requested length are what stand between the attacker and kernel physical pages.
- (d) The kernel path: mmap → ksys_mmap_pgoff → vm_mmap_pgoff → do_mmap → mmap_region → get_unmapped_area → file->f_op->mmap. The driver handler receives a struct vm_area_struct * whose vm_pgoff, vm_start, vm_end, vm_flags and vm_ops are all attacker-influenced.
- (e) Bug patterns regularly found in mmap:

- i. Missing bounds checks: `remap_pfn_range` or `vm_insert_page` called with an offset or size that was never validated against the device's own memory region.
 - ii. Integer overflows in sizes: `offset + size` computed in 32 bits where either operand can be attacker-chosen, so a wrapped sum passes the check while the real range does not (the `model_driver.c` companion in the Linux kernel module reproduces exactly this shape).
 - iii. Out-of-bounds accesses through the `vm_operations_struct` attached to a mapped region: a `.fault` or `.access` handler that trusts `vm_pgoff` without re-checking it, or an `.open`/`.close` handler with a lifetime bug.
 - iv. Missing `VM_IO`/`VM_PFNMAP`/`VM_DONTEXPAND` flags, so the resulting mapping can be `mremapped`, `forked` or written in ways the driver did not intend.
 - v. Page-alignment mistakes: the offset of a file-backed mapping must be page-aligned; a driver that scales `vm_pgoff` by its own page size without checking can index outside its region.
 - vi. Races with `munmap`/`mremap`/`fork` (CVE-2016-5195, Dirty COW, is the canonical private-mapping race).
- (f) Reference: <https://labs.f-secure.com/assets/BlogFiles/mwri-mmap-exploitation-whitepaper-2017-09-18.pdf>

3. Explain `ptrace`. How might you inject code into another process using `ptrace`?

- (a) `ptrace` is a syscall (the entry point `sys_ptrace` is implemented in `kernel/ptrace.c`, with the per-architecture request handlers in `arch/*/kernel/ptrace.c`) that lets one user-space process trace and debug another. Enforcement of DAC/MAC permissions happens in the kernel before the attach succeeds: DAC through `ptrace_may_access()` and MAC through the LSM `security_ptrace_access_check` hook (plus Yama's `ptrace_scope` on many distributions).
- (b) The interface can attach or stop a process, inspect and modify its virtual memory and registers, and single-step or syscall-stop it. The request set worth knowing: `PTRACE_ATTACH`/`PTRACE_DETACH`, `PTRACE_SEIZE`/`PTRACE_INTERRUPT` (attach without stopping, stop on demand), `PTRACE_PEEKTEXT`/`PTRACE_PEEKDATA`, `PTRACE_POKETEXT`/`PTRACE_POKEDATA`, `PTRACE_GETREGS`/`PTRACE_SETREGS`, `PTRACE_GETREGSET`/`PTRACE_SETREGSET` (portable, per-architecture register sets), `PTRACE_CONT`, `PTRACE_SINGLESTEP`, `PTRACE_SYSCALL` and `PTRACE_SETOPTIONS` (which enables `PTRACE_O_TRACESYSGOOD`, `PTRACE_O_EXITKILL`, `PTRACE_O_SUSPEND_SECCOMP` and the rest of the event options).
- (c) Injection sequence (the standard code-injection walk):
 - i. `PTRACE_ATTACH`, or `fork + exec` with `PTRACE_TRACEME`; stopping the group leader is ideal because it stops the whole thread group.
 - ii. For a multi-threaded target, attach to each thread individually — the tracer must enumerate `/proc/<pid>/task`.
 - iii. Back up the current page at the program counter using `PTRACE_PEEKTEXT`.
 - iv. Back up the register state with `PTRACE_GETREGS` (or `PTRACE_GETREGSET`).
 - v. Overwrite the executing page with shellcode using `PTRACE_POKETEXT`.
 - vi. `PTRACE_CONT` the thread.

- vii. Have the shellcode trap when it is done (`int3` on x86, `brk` on ARM64) so the tracer regains control.
 - viii. Restore the original page and register state.
 - ix. `PTRACE_DETACH`.
 - (d) Alternatives that need no `ptrace` attach at all, but the same permission check: `process_vm_readv/process_vm_writev` and reads/writes to `/proc/<pid>/mem`. Debuggers use these because they are far faster than `PTRACE_PEEKDATA` word-by-word, and `gdb`'s `call` command uses exactly this path to run a function inside the inferior.
 - (e) Android specifics: SELinux gates `ptrace` between domains, so a normal `untrusted_app` cannot attach to another app or to `system_server`. The practical routes are a `userdebug/eng` build with `adb root`, a debuggable application (`android:debuggable="true"`), or the `run-as` path for a debuggable package. `adbd` running as `shell` can attach to processes the `shell` domain is allowed to reach, which is why `gdbserver` is started as `root` in the standard recipe below.
4. Describe how SELinux policies are used. How would you check the context of a file system object and a process? What file systems support SELinux and which do not?
- (a) SELinux is a mandatory access control (MAC) policy enforced at the syscall gateways and other chokepoints in the kernel. The kernel exposes a consistent interface for any security module (the LSM hooks covered in the kernel lecture), so SELinux, SMACK and AppArmor all enforce through the same call sites.
 - (b) The current process context lives in `/proc/self/attr/current` — read that file (or `id -Z` on a host, `ps -Z` for every process). A context has the form `user:role:type:level`, e.g. `u:r:untrusted_app:s0:c512,c768`. Bionic (Android's libc) writes to these files to set policy knobs, and `setexeccon/setcon` move a process between domains.
 - (c) Transitions are the part that matters: a `type_transition` rule says which domain a process enters when it executes a file. The classic Android example is `init` entering the `zygote` domain and `zygote` forking into `untrusted_app` for an app; an audit of a service is incomplete without the domain it runs in.
 - (d) File-system objects: query with `ls -Z`, `stat`, or directly with the `security.selinux` extended attribute (`getfattr -n security.selinux`). The expected context for a path comes from `file_contexts`, and `restorecon` re-applies it.
 - (e) AVC denials (the enforcement events) are logged to the kernel ring buffer and the audit log: `dmesg | grep avc` or `/data/misc/audit/audit.log`. `getenforce/setenforce` show and relax the global mode (permissive mode logs but does not deny, and requires root).
 - (f) File systems: `ext4` (Android's principal file system) supports extended attributes and therefore native SELinux labeling; so do `f2fs`, `xfs` and `btrfs`. `vfat/exfat` have no extended attributes, so the Android kernel compensates with hard-coded contexts supplied by the FUSE `sdcard` daemon (formerly the `sdcardfs` kernel shim) and with `genfs_contexts` entries, or with the `context=` mount option for removable media.

- (g) Android specifics: SELinux is enforcing by default since Android 5.0 (permissive by default from 4.3), the policy is split into `plat/vendor/product` partitions, and `neverallow` rules constrain what a vendor policy can grant. Because it is a policy, not a bug boundary, a driver reachable by a labeled domain is reachable: SELinux changes who can reach the surface, never whether the surface has bugs.

5. What do you initially look for when auditing Linux source?

- (a) Custom vendor code that is not in the mainline, especially drivers. The mainline is fuzzed and audited for years; the vendor drop is not.
- (b) Custom `mmap` code, for the reasons in Question 2 above.
- (c) Custom `ION/DMA-BUF` ioctls and allocator interfaces, where every new handle type is a new parser with new size and flag arithmetic.
- (d) Inconsistent code quality or code style — a strong signal that a piece of code was written by a different team, under different review, at a different time.
- (e) Excessive comments that may signal imprecise logic; a comment explaining what a check is supposed to do is a good place to check whether it does it.
- (f) Interfaces between large subsystems, and consistency between the two sides of each interface: the `compat_ioctl` path for a 64-bit kernel with a 32-bit caller is the classic mismatch.
- (g) The 32-bit compatibility paths, uninitialized stack or heap structures handed back with `copy_to_user`, and unchecked `copy_from_user` return values are the recurring shapes in vendor drivers.

Part II: Android-Specific Auditing

1. What are the differences between Android and Linux distributions?

- (a) Android adds, on top of a Linux kernel:
 - i. Binder IPC — the process-to-process RPC and object model, plus the driver's own attack surface (CVE-2019-2215 was a Binder use-after-free).
 - ii. Ashmem shared memory (historically, an anonymous shared-memory allocator; deprecated in favour of `memfd` on newer kernels).
 - iii. ION allocators (historically, a vendor memory allocator exposed to userspace; deprecated in favour of `DMA-BUF` heaps).
 - iv. TrustZone drivers — the userspace-visible half of the ARM secure world interface.
 - v. Trustlets — the small applications running inside the secure world.
 - vi. Android DSP RPC (the `adsprpc` drivers on Qualcomm platforms) — remote procedure calls into the DSP.
 - vii. `dm-verity` for system integrity checking at boot, and Android Verified Boot (AVB) above it.
 - viii. Android-specific security mechanisms: Knox and TIMA and Real-time Kernel Protection (RKP) on Samsung, `cryptfs` and file-based encryption (FBE), plus the `keystore/keymaster/gatekeeper` services.

- ix. The Android userspace itself: `zygote`, `system_server`, `bionic` (not `glibc`), the HALs (HIDL/AIDL), and the hardware abstraction layers behind `hwservice manager/service manager`.
 - (b) Consequences for the auditor: the vendor half of the tree is written by a different team, reviewed by a different process and often not fuzzed at all. The vendor additions are the audit queue, exactly as in the kernel module.
2. **Walk through the process of debugging a binary on the phone.** (Some of these steps can be reordered.)
- (a) Root the phone if you need to debug a system application (`vold`, `system_server`, `init`, etc). On a `userdebug` or `eng` build `adb root` is enough; on a production build the phone must be rooted first (Question 4).
 - (b) Use `adb` to push `gdbserver` onto the phone — preferably the `gdbserver` included in the NDK, with the API version matching the phone.
 - (c) `adb` forward the TCP port you will use for the GDB remote protocol:

```
1 $ adb forward tcp:1234 tcp:1234
```
 - (d) Attach `gdbserver` to the target process:

```
1 $phone gdbserver --attach :1234 <pid>
```
 - (e) From the host, connect with architecture-appropriate `gdb` (prefer `gdb-multiarch` on Ubuntu):

```
1 $host gdb-multiarch
2 > target remote :1234
```
 - (f) Point the debugger at the target's own libraries so symbols and shared-library addresses resolve:

```
1 > set sysroot /tmp/pulled-system
2 > set solib-search-path /tmp/pulled-system/lib64
3 > sharedlibrary
```
 - (g) The LLDB equivalent, which is what Android Studio's native debugger uses under the hood:

```
1 $ lldb-server platform --server --listen *:1234      # on the device
2 $ lldb
3 > platform select remote-android
4 > platform connect connect://<host>:1234
5 > process attach --pid <pid>
```
 - (h) For an application rather than a daemon, a debuggable build and `run-as <package>` avoid the root requirement entirely.
3. **What types of problems are typical when debugging remotely with `gdbserver`?**
- (a) `gdb` infers ARM/AArch64 from the loaded executable, but on 32-bit systems it cannot tell whether the processor is in ARM or Thumb mode — set the mode explicitly (`set arm force-mode thumb`). This may require reversing the code to determine which mode a particular text address runs in.

- (b) gdb is sensitive to the debugger version pairing: match the gdbserver version against the host gdb/gdb-multiarch. A mismatch shows up as Remote 'g' packet reply is a truncated register set, or an inability to read memory at all.
- (c) Missing symbols: a stripped system library yields meaningless frames. Pull the target's libraries to the host and set sysroot/solib-search-path, or use ndk-stack against a tombstone for the crash path.
- (d) Permissions: SELinux may deny the attach, and ptrace_scope may forbid attaching to a non-child. setenforce 0 on a userdebug build, or attach as root, resolves most of these.
- (e) ASLR and timing: the address layout differs between runs, and the remote stub perturbs timing enough that race bugs can disappear or appear under the debugger.

4. Walk through typical ways to root Android phones.

- (a) Open developer mode by repeatedly clicking Build Number in the About Phone settings.
- (b) OEM-unlock the bootloader using the Developer mode menu (fastboot flashing unlock, or the vendor-specific fastboot oem unlock); this usually wipes userdata.
- (c) Install TWRP via fastboot (or Odin on Samsung, Flashtool on MediaTek), which is usually vendor-specific. Booting the recovery image directly with fastboot boot twrp.img avoids flashing it at all.
- (d) Boot into recovery mode, which brings up TWRP.
- (e) Install SuperSU or Magisk. Modern systemless root (Magisk) patches the boot ramdisk rather than the system partition, which is what survives dm-verity and OTA updates.
- (f) Reboot and su.

5. What problems could you run into along the way when rooting a phone?

- (a) Bootloader unlock blocked by a carrier lock (Verizon, AT&T), or by the OEM-unlock waiting period (some vendors enforce a multi-day delay before the toggle becomes usable).
- (b) TWRP misbehaving, sometimes with no error message at all; on A/B devices the wrong slot can be the culprit.
- (c) dm-verity: try the no-verity option from TWRP, or flash a vbmeta image with verification disabled.
- (d) File-based encryption: the userdata partition may have to be formatted before root will boot, which loses all data.
- (e) Vendor-specific lock state: RMM/KG (Knox Guard) "prenormal" state on Samsung blocks the unlock until the device has been online for the required period.
- (f) Integrity attestation: SafetyNet (now Play Integrity) detects the unlocked bootloader and the root itself, which matters for applications that check it.
- (g) If the phone cannot be permanently rooted, a TempRoot — a fancy name for an exploit — is the fallback. Hopefully one exists for the target version.

- (h) Many other random things, including OTA updates silently reverting a root and the device entering a boot loop after a bad Magisk module.

6. With an adb session on the phone, what is the attack surface to reach the kernel?

- (a) Kernel drivers with `/proc` entries — write and `ioctl` handlers registered through `proc_create` are reachable with only the DAC mode bits and the SELinux label as a gate.
- (b) Kernel drivers exposing `/dev` entries — the `file_operations` verbs of each character device.
- (c) `debuggerd/crash_dump` processing of executables — a parser of attacker-authored crash state.
- (d) GPUs: Mali and `kgs1` (Adreno) — the largest vendor-written surface on a modern phone.
- (e) Special file-system operations, for example the `cryptfs` functions and the FUSE-based storage daemon.
- (f) Daemons listening on UNIX domain sockets — `vold`, `netd`, `installd`, `zygote` and the vendor HALs.
- (g) Android services, especially those that do not check root or are vendor-provided — Binder services reachable from a `shell` or app context.
- (h) Binder itself — CVE-2019-2215 is the canonical Binder use-after-free, and the driver remains a high-value target.
- (i) TrustZone drivers (`qseecom` and friends) — the userspace-visible half of the secure world interface.
- (j) Kernel helper interfaces reachable from a shell: `keyctl`, `bpf`, `io_uring` on newer kernels, `userfaultfd` and `perf` events, as listed in the kernel module.

7. If you could not get adb, what is the attack surface for initial access?

- (a) USB: the device mounts as a mass-storage device, or enumerates as a host USB keyboard/mouse (a malicious peripheral can inject input), or as Ethernet-over-USB.
- (b) The cell baseband processor — see the baseband research by Amat Cama; the modem parses attacker-authored frames from the network before any userspace runs.
- (c) Bluetooth — BlueBorne (CVE-2017-1000251 and its siblings) and BlueFrag (CVE-2020-0022) are the reference examples.
- (d) Near-field communication (NFC) — tag and peer-to-peer parsing, historically a source of memory-corruption bugs.
- (e) Wi-Fi — the supplicant and the Broadcom/Qualcomm firmware beneath it (Broadpwn, CVE-2017-9417) are reachable from radio range.
- (f) Over-the-air messaging: MMS and the media parsers behind it, the surface that carried Stagefright (CVE-2015-3864).

8. How does debuggerd/crash_dump work?

- (a) Before Android 8 (Oreo): `debuggerd` was a daemon and any process could forward its pid to it over a UNIX domain socket. Bionic installed signal handlers via the dynamic linker; on `SIGSEGV` the handler sent the pid to `debuggerd`, which (having `ptrace` ability) stopped the process, attached to it and wrote a tombstone file.
- (b) Android 8 and above: the `debuggerd` daemon is gone. The linker installs the signal handlers, which invoke `/system/bin/crash_dump32` or `/system/bin/crash_dump64`. When this is launched, it attaches to the crashing process and creates the tombstone, brokered through the `tombstoned` daemon so that only one crash is collected at a time.
- (c) The tombstone is written under `/data/tombstones/tombstone_XX` and contains the thread list, registers, the memory map and a stack dump. `ndk-stack` (or `debuggerd -b <pid>`) symbolicate it against the matching `symbols` directory.
- (d) Related surfaces in the same pipeline: ANR traces (a `SIGQUIT` sent to an unresponsive process dumps Java thread stacks to `/data/anr/`), dropbox entries under `/data/system/dropbox`, and logcat tags such as `DEBUG` and `tombstoned`.

9. How do you emulate Android?

- (a) The Android-x86 project is usable: it runs in VMware/QEMU/VirtualBox, and you can compile custom kernels and AOSP — <https://www.android-x86.org/>.
- (b) AOSP's own virtual device (cuttlefish, driven by `cvd`) runs the real system images on QEMU or CrosVM, which is the closest thing to a stock Android target that can still be instrumented.
- (c) The SDK emulator (emulator with an AVD) supports `-writable-system` for a writable system partition, `-show-kernel` for kernel logs, and `-qemu -s -S` to attach a kernel debugger through the same GDB remote protocol as the debugging module.
- (d) Container-based approaches (Waydroid, Anbox) run the Android userspace against the *host* kernel, which makes them useless for kernel auditing but convenient for userspace work.
- (e) The downside of all emulation: you lose the vendor-specific code and TrustZone entirely, the kernel is a generic goldfish one, and the baseband and GPU are not the real thing. Kernel debugging on a physical phone, by contrast, is hard to obtain, which is why emulation is the standard first target.

How This Module Maps to the Course

1. The kernel lecture's driver-auditing discipline is applied here to Android's vendor surfaces (Binder, `ashmem`, `ION/DMA-BUF`, GPU drivers); the debugging module's `remote-gdb` section is realized as the `gdbserver` walk-through above.
2. The audit-queue rule stands: mainline code is comprehensively fuzzed by the distribution; the vendor additions (`adsprpc`, `kgs1`, `trustlets`) are where the unaudited parsing lives.

3. The source auditing module's attack surface analysis supplies the enumeration method; this module supplies the phone-specific reachability rules (DAC, SELinux label, namespace) that decide which of those surfaces an adb shell can actually touch.
4. The fuzzing module's harness work is the natural next step: each surface listed in Part II is a candidate harness, and each of the drivers named is a candidate for a syzkaller-style descriptor.

References

1. android_questions.txt in this repo (source of parts I–II; keep in sync)
2. mwr infosecurity mmap exploitation whitepaper (linked above)
3. Google Project Zero: Binder (CVE-2019-2215) and Stagefright writeups
4. Android security bulletins and the AOSP source (android.googlesource.com)
5. The Linux kernel module and its references for LSM/DAC/MAC mechanics.
6. Android Open Source Project, "Verified Boot" and "dm-verity" documentation
7. F-Secure mmap exploitation whitepaper and the Linux auditing notes